

# ServiceNow Incident Management – Client Script & UI Policy

## Project Documentation

|                   |                                     |
|-------------------|-------------------------------------|
| Team ID           | SWTID-2026-2035                     |
| Team Size         | 5                                   |
| Project Execution | 28 September 2026 – 01 October 2026 |
| Document Date     | 03 October 2026                     |

### 1. Introduction

- Project Title: Implement Client Script & UI Policy (Incident)
- Team Leader: Ahil R
- Team Members: Mohan Das K J; DILIBAN A; Shiva Prakash K R; Venkatraghul M
- Platform: ServiceNow Developer Instance
- Project Goal: Configure and validate client-side behavior on the ServiceNow Incident form using Client Scripts and UI Policy rules.

### 2. Project Overview

The project implements controlled and validated Incident-form behavior in ServiceNow. The solution uses UI Policy and Client Script mechanisms so that Incident fields respond consistently to configured conditions before a record is saved.

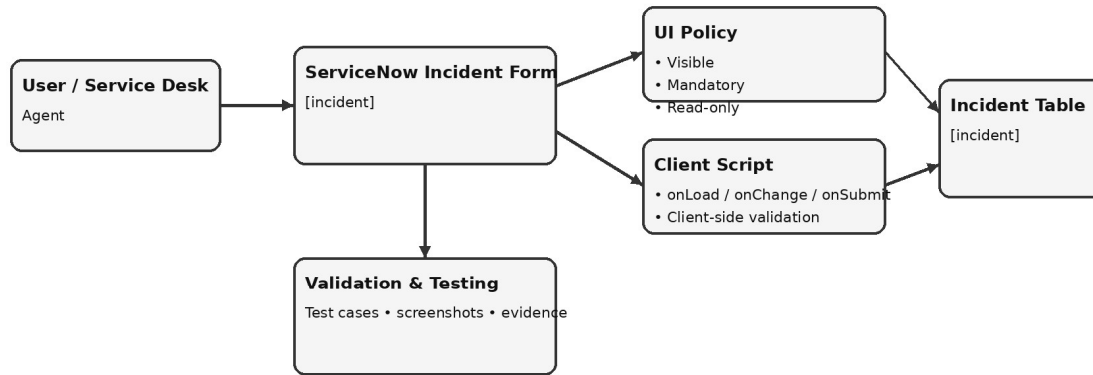
Key features:

- UI Policy control of field visibility, mandatory state, and read-only state.
- UI Policy Action for controlling the configured Incident field behavior.
- onChange Client Script for dynamic client-side validation when relevant field values change.
- onSubmit Client Script for validation before an Incident record is saved.
- onCellEdit validation for controlled list-edit interactions where applicable.
- Positive, negative, reverse-condition, form-save, and list-edit test scenarios with screenshot evidence.

### 3. Architecture

The solution is implemented inside the ServiceNow platform rather than as a separate MERN application. The Incident form is the main user interface, while UI Policy and Client Script logic provide the client-side control and validation layer. The resulting record is stored in the standard ServiceNow Incident table.

## ServiceNow Incident - Client Script & UI Policy Architecture



Flow: user interacts with Incident form → UI Policy controls field state → Client Script performs client-side logic → validated data is submitted to the Incident table

Evidence: configured records, test results, and screenshots document the behavior implemented in the ServiceNow developer instance.

Figure 1: Project-specific ServiceNow Incident – Client Script & UI Policy architecture.

- User Interface: ServiceNow Incident form used by the service-desk/agent.
- Client-side rules: UI Policy, UI Policy Action, onChange, onSubmit, and onCellEdit Client Scripts as configured for the required behavior.
- Data layer: Standard ServiceNow Incident [incident] table.
- Validation: Test cases and screenshots verify configured behavior before and during record submission.

## 4. Setup Instructions

Prerequisites:

- Access to the assigned ServiceNow Developer Instance.
- A ServiceNow user account with the required developer/configuration permissions.
- Access to the Incident application and form.
- A browser capable of running the ServiceNow web interface.

Configuration steps:

1. Open the assigned ServiceNow Developer Instance.
2. Navigate to the Incident application and open the Incident form.
3. Create or verify the required UI Policy and UI Policy Action configuration.
4. Create or verify the required Client Script records and their applicable field/condition settings.
5. Save the configuration and test the behavior on the Incident form.
6. Capture evidence for the configured rules and test results.

## 5. Configuration / Structure

Because the project is implemented directly in ServiceNow, there is no conventional React client/server folder structure. The logical project structure is:

- Incident Form – primary user-facing form for creating and updating incidents.
- UI Policy – controls field state based on configured conditions.
- UI Policy Action – applies mandatory, visible, or read-only behavior to selected fields.
- Client Scripts – contain client-side logic for onChange, onSubmit, and applicable onCellEdit interactions.
- Incident Table – stores the validated Incident records.
- Test Evidence – screenshots and test results demonstrating expected behavior.

## 6. Running the Application

The solution runs in the ServiceNow Developer Instance; no local npm start command is required.

1. Sign in to the assigned ServiceNow Developer Instance.
2. Open the Incident application/form.
3. Create a new Incident or open an existing test Incident.
4. Change the relevant field values and observe the configured Client Script and UI Policy behavior.
5. Attempt to save the record and verify the configured validation and field-state rules.
6. Repeat the applicable scenarios for reverse conditions and list editing.

## 7. API Documentation

No separate custom REST API is exposed by this project. The implementation uses ServiceNow's native Incident form, client-side configuration records, and platform data services.

- Primary data object: Incident [incident] record.
- Client interaction: Incident form field changes and form submission.
- Persistence: ServiceNow platform saves validated Incident data to the Incident table.
- The project does not require a separate Node.js/Express backend or MongoDB database.

## 8. Authentication

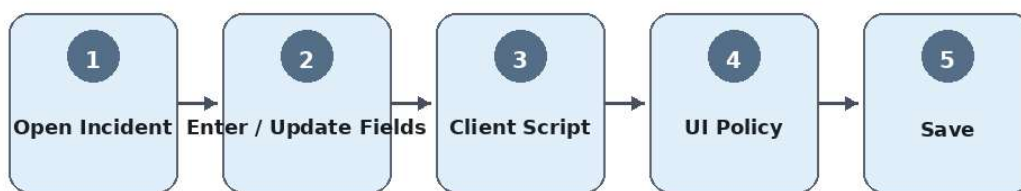
Authentication is handled by the ServiceNow platform through the assigned developer-instance account and session. Authorization is determined by the user's ServiceNow roles and permissions. Project credentials are intentionally not stored in this document.

- Access is restricted to authorized project users.
- Configuration and testing activities require the appropriate ServiceNow developer/configuration permissions.
- The document contains no passwords, tokens, or private credentials.

## 9. User Interface

The primary interface is the ServiceNow Incident form. The configured UI Policy changes field state according to its conditions, while Client Scripts perform dynamic validation and submission checks.

### Incident Form Workflow



Agent opens an Incident record → User changes relevant JavaScript field → Client Script validates client-side conditions → UI Policy / read-only view applied → Incident data is saved.

Result: the Incident form applies the configured client-side behavior before the record is committed.

Figure 2: Incident form workflow used for the project.

- Open Incident → enter/update fields → Client Script validation → UI Policy behavior → save.

- Expected result: configured client-side behavior is applied before the Incident record is committed.

## 10. Testing

Testing covered the configured Incident form behavior and included both positive and negative scenarios.

- Client Script execution on relevant field changes.
- UI Policy conditions and corresponding field state changes.
- Mandatory, read-only, and visibility behavior.
- Form validation and save behavior.
- Reverse-condition testing to confirm rules do not remain active outside their intended conditions.
- List-edit/onCellEdit behavior where configured.
- Evidence capture through screenshots from the ServiceNow Developer Instance.

Testing period: 29 September 2026 – 01 October 2026.

## 11. Screenshots or Demo

The working demonstration is the configured ServiceNow Developer Instance. Project evidence consists of screenshots showing the configuration records, Incident form behavior, and validation results.

- Demo environment: Assigned ServiceNow Developer Instance – Incident application/form.
- Evidence: configuration and execution screenshots collected during the project.
- The team's captured screenshots should be used as the primary evidence of completion.

## 12. Known Issues

- The configured behavior depends on the Incident fields, conditions, and roles available in the assigned ServiceNow instance.
- Client-side rules guide and validate user interaction on the Incident form; they are not a replacement for server-side controls where stronger data enforcement is required.
- Behavior may differ if the configuration is copied to an instance with different field definitions, UI policies, scripts, or permissions.

## 13. Future Enhancements

- Extend the same validation pattern to additional Incident fields and related record types.
- Add complementary server-side validation where stronger data enforcement is required.
- Introduce automated regression testing for the configured Client Scripts and UI Policies.
- Package the configuration for controlled promotion between development, test, and production instances.
- Add reporting or dashboard views to monitor Incident data quality and rule effectiveness.

Prepared for Team ID SWTID-2026-2035 | Document Date: 03 October 2026